

CSE3443 SOFTWARE MAINTENANCE AND EVOLUTION

CSE3443 Course Contents

- **Topic 1: Software Maintenance**
- Topic 2: Key Issues in Software Maintenance
- Topic 3: Evolution and Maintenance Model
- Topic 4: Techniques for Maintenance
- Topic 5: Software Maintenance Tools

Topic 1: Software Maintenance

1. Definitions and Terminology
2. Nature of Maintenance
3. Need for Maintenance
4. Majority of Maintenance Costs
5. Evolution of Software
6. Categories of Maintenance



OUTLINE OF TODAY'S LECTURE (Week 1)

1. **Introduction to Software Maintenance and Evolution – real world issues**
2. **Topic 1: Software Maintenance (Part 1)**

1. Definitions and Terminology

2. Nature of Maintenance
3. Need for Maintenance
4. Majority of Maintenance Costs
5. Evolution of Software
6. Categories of Maintenance

Real world examples



In 2018, Facebook experienced a major software bug that exposed the private information of millions of users.

The bug was caused by a software maintenance issue where Facebook developers altered the code of a feature called "View As," enabling users to view their profiles as if they were someone else.

This modification inadvertently introduced a vulnerability (bug) that enabled attackers to pilfer access tokens, granting unauthorized access to user accounts without requiring a password.

Although Facebook swiftly addressed the bug, it raised apprehensions regarding the security and dependability of Facebook's software infrastructure.

Do you know about the Facebook new bug?

2018

- <https://www.youtube.com/watch?v=H5s5kSdXW9Q>

Introduction to Software Evolution and Maintenance

- Software Maintenance is hard
- Even after deployment, software systems may need to undergo changes, for example to fix problems or improve the system.
- This activity is called “software maintenance”.
- Software maintenance is crucial, but critical, activity in the life cycle of a system.
- It's often harder to maintain a system than to develop it.
- But it's even harder to design a maintainable system, because it's hard to foresee all future changes.

Warming up activity 😊

- Software maintenance is crucial, but critical, activity in the life cycle of a system.
 - Software Maintenance is hard
 - Even after deployment, software systems may need to undergo changes, for example to fix problems or improve the system.
- In which cycle the software maintenance is?
 - Why ya?
 - What kinds of activities that are performed during maintenance?

1. Definitions and Terminology

- **Definition:**

1. Software maintenance refers to the process of modifying, updating, and improving software applications to ensure that they continue to meet the changing needs of users.
2. Modification of a software product after delivery to correct faults, to improve performance or other attributes, or to adapt the product to a modified environment.

Software maintenance

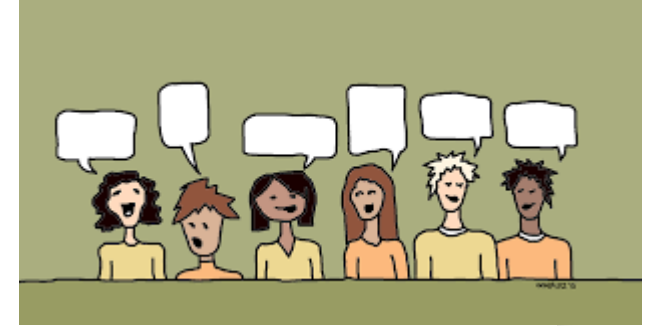
- Modifying a program after it has been put into use.
- The term is mostly used for changing **custom software**.
- **Generic software** products are said to evolve to create new versions.
- Maintenance does not normally involve major changes to the system's architecture.
- Changes are implemented by modifying existing components and adding new components to the system.



Why Maintenance is Crucial

- Software maintenance is critical to ensuring that software remains reliable, secure, and functional over time.
- It helps to identify and fix bugs, add new features, and enhance performance.
- Cost Savings:
 - By investing in maintenance, organizations can reduce the total cost of ownership of software, as they can avoid the need for expensive upgrades or replacements.
- User Satisfaction:
 - Maintenance can also improve user satisfaction, as it enables the software to evolve over time and meet the changing needs of users.
- Business Success:
 - Finally, maintenance is crucial to the long-term success of any software application, as it enables organizations to stay competitive and responsive to changing market conditions.

What could be the causes of software maintenance issues?



- Relate to your experience in developing your own PITA system.
- Let's assume that you've finished with the implementation, you've deployed it to your clients and users.
- Now, it's time to maintain – do some changes to the system.
- However, maintenance has become a problem
 - i.e. difficult to maintain, many bugs reported
- What could be the causes?

Group discussion

- Common causes to software maintenance problems:

1. **Lack of Traceability**
2. **Lack of Code Comments**
3. **Incomplete or unclear requirements**
4. **Lack of testing**

Common causes of maintenance issues

- **Lack of Traceability**

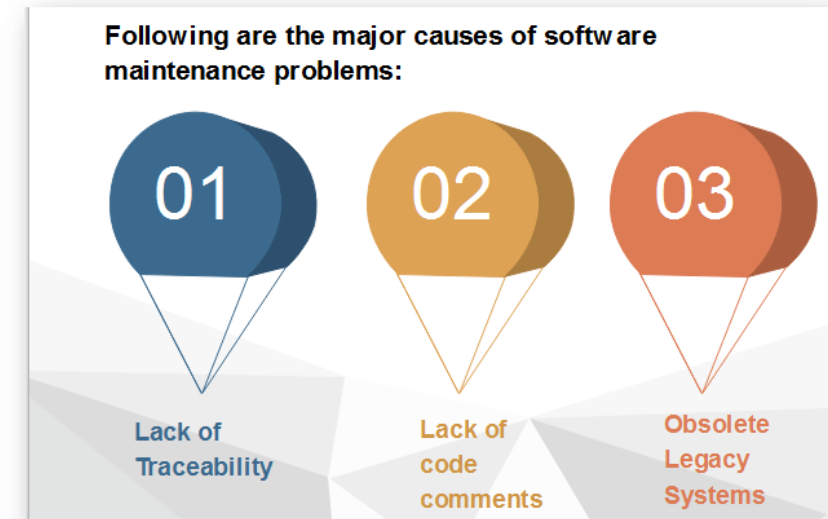
- Codes are rarely traceable to the requirements and design specifications.
- It makes it very difficult for a programmer to detect and correct a critical bug affecting customer operations.
- Like a detective, the programmer pores over the program looking for clues.
- Life cycle documents are not always produced even as part of a development project.

- **Lack of Code Comments**

- Most of the software system codes lack adequate comments.
- Lesser comments may not be helpful in certain situations.

- **Incomplete or unclear requirements:**

- Incomplete or unclear requirements.
- When requirements are unclear or incomplete, it can be difficult for software engineers to understand what the software is supposed to do, leading to errors, bugs, and other quality issues.



Other common causes of maintenance issues

- poor documentation
 - insufficient resources
 - inadequate project management (including low productivity, low motivation, low skill levels)
-
- Effective software maintenance requires attention to detail, a commitment to quality, and a focus on continuous improvement.

- **Code maintainability** is one of the most important elements of any application.

- a metaphor for sloppy, unmaintainable code.



```
public abstract class AbstractCollection implements Collection {
    public void addAll(AbstractCollection c) {
        if (c instanceof Set) {
            Set s = (Set)c;
            for (int i=0; i < s.size(); i++) {
                if (!contains(s.getElementAt(i))) {
                    add(s.getElementAt(i));
                }
            }
        } else if (c instanceof List) {
            List l = (List)c;
            for (int i=0; i < l.size(); i++) {
                if (!contains(l.get(i))) {
                    add(l.get(i));
                }
            }
        } else if (c instanceof Map) {
            Map m = (Map)c;
            for (int i=0; i<m.size(); i++)
                add(m.keys[i], m.values[i]);
        }
    }
}
```

Duplicated Code (points to the `if (c instanceof Set)` and `if (c instanceof List)` blocks)

Duplicated Code (points to the `if (!contains(s.getElementAt(i)))` and `if (!contains(l.get(i)))` conditions)

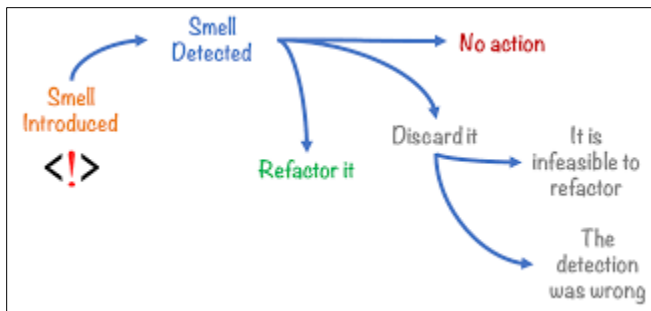
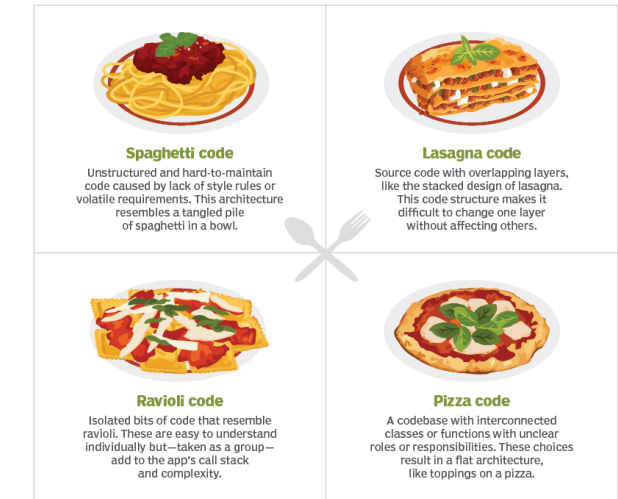
Alternative Classes with Different Interfaces (points to the `if (!contains(s.getElementAt(i)))` and `if (!contains(l.get(i)))` conditions)

Switch Statement (points to the `if (c instanceof Set)`, `if (c instanceof List)`, and `if (c instanceof Map)` blocks)

Inappropriate Intimacy (points to the `m.keys[i]` and `m.values[i]` access in the Map block)

Long Method (points to the entire `addAll` method)

The pasta theory of programming



REFACTORING TECHNIQUE

What is Refactoring?

- <https://www.youtube.com/watch?v=-BSQZEX0kpg>

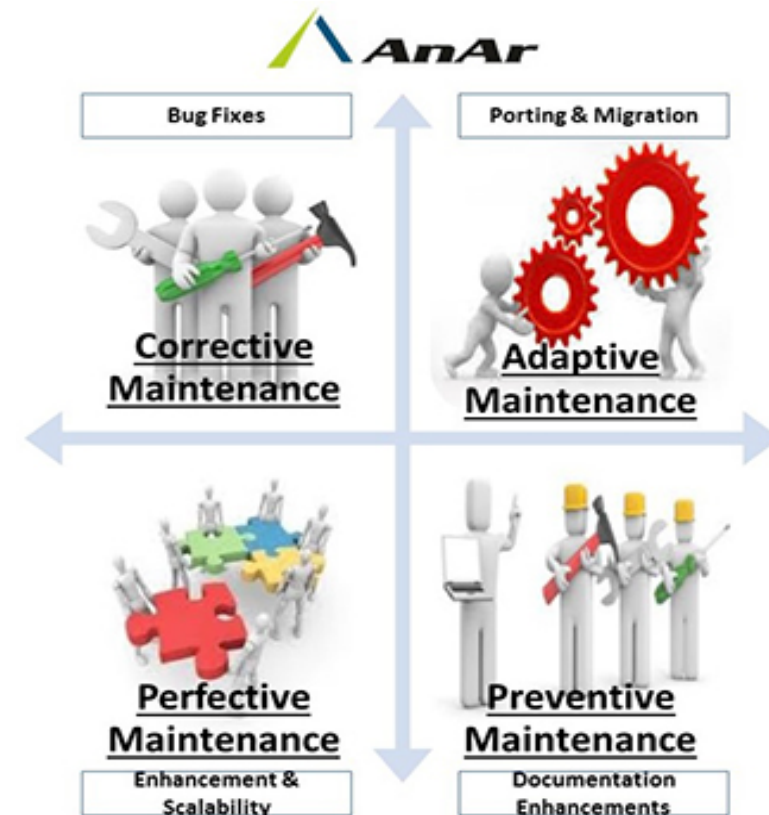
2. Nature of Maintenance

- Types of Maintenance: Corrective, Adaptive, Perfective, Preventive

Types of Software Maintenance

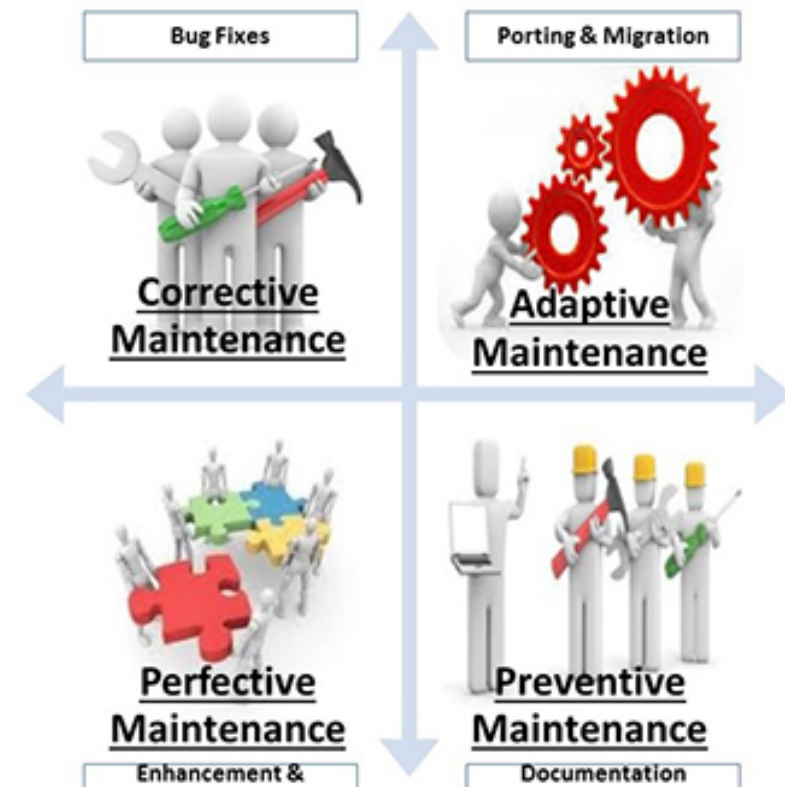
- Different types of maintenance can be distinguished, depending on the goal of the maintenance activity:

- Adaptive maintenance
- Corrective maintenance
- Perfective maintenance
- Preventive maintenance



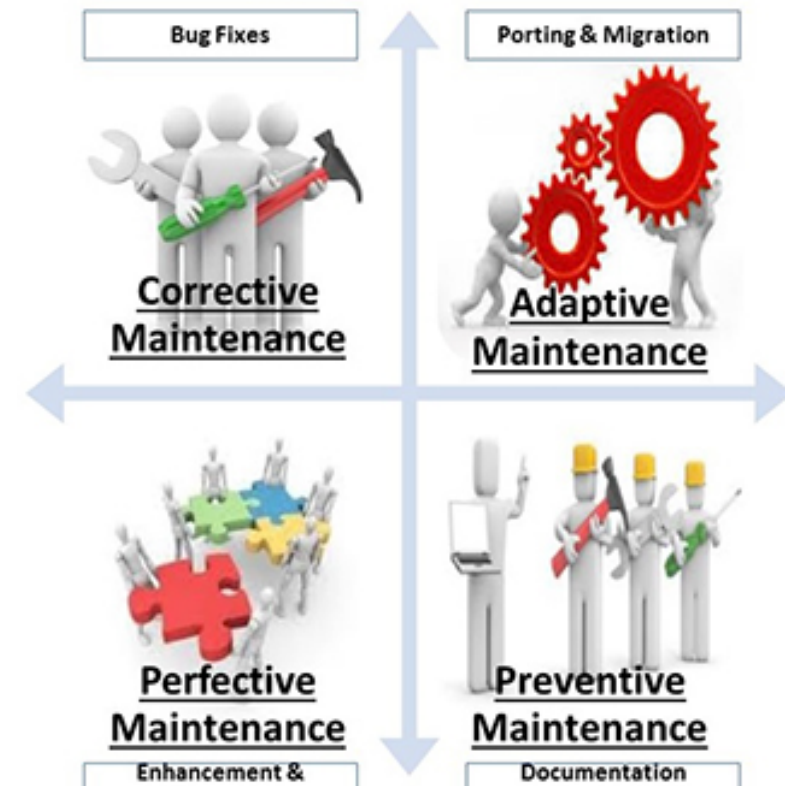
Adaptive Maintenance

- Modification of a software product performed after delivery to keep a computer program usable in a changed or changing environment.
- Occurs when, as a result of external influences or strategic changes, a software system needs to be adapted to remain up to date.
- Examples:
 - The Malaysia government decides to change from GST (goods and services tax) to SST (sales and service tax) in Sep 2018.
 - A company decides to introduce an online system to allow clients to access its services.



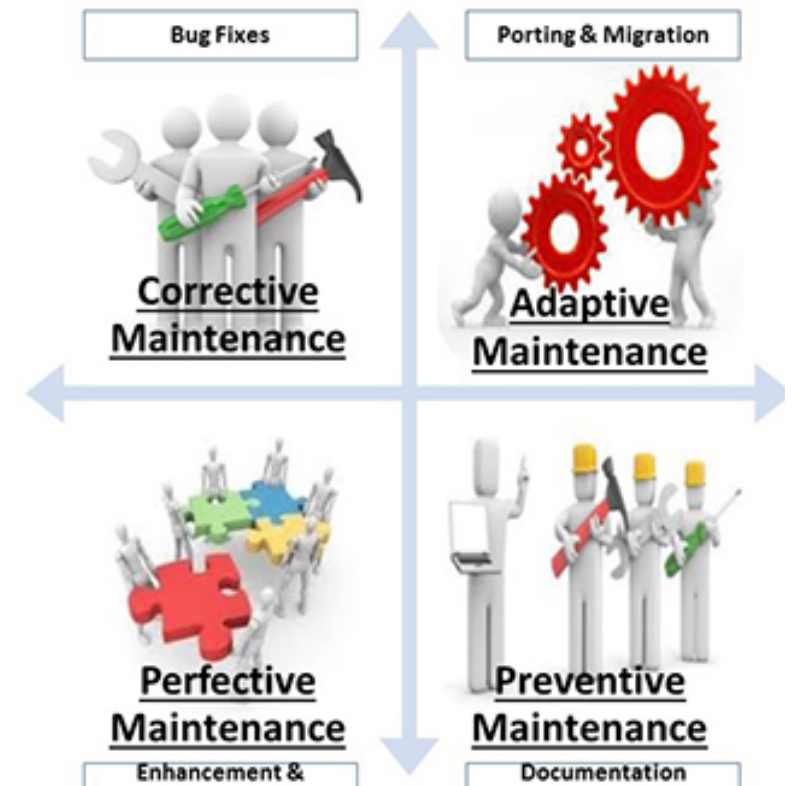
Corrective Maintenance

- Reactive modification of a software product performed after delivery to correct discovered faults.
- Often occurs after deployment when customers detect problems that were not discovered during initial testing of the system.
- These errors should be fixed.



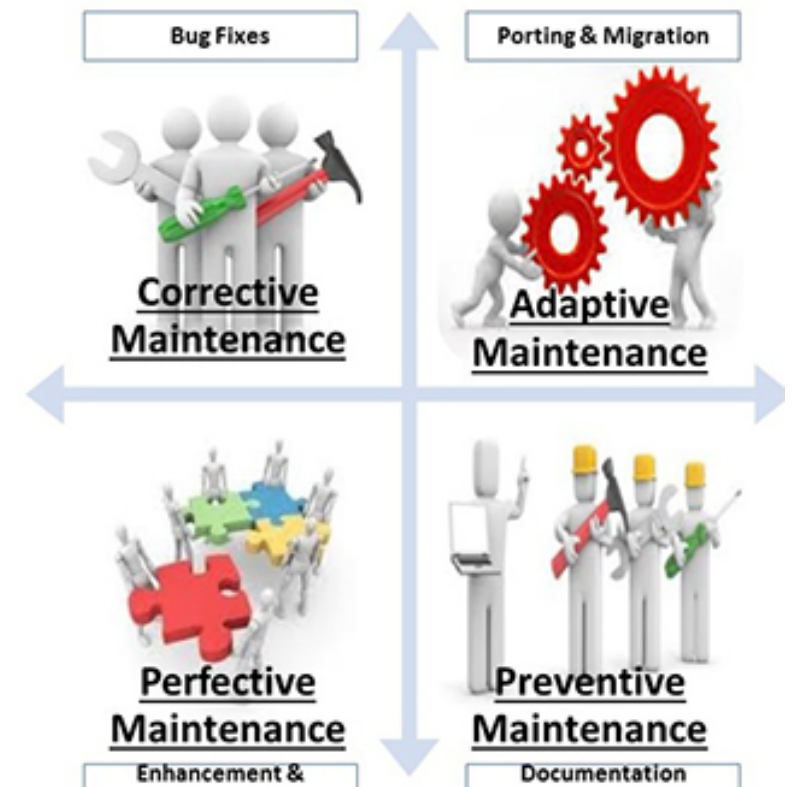
Perfective Maintenance

- Improving the quality of a software system.
- Modification of a software product after delivery to improve performance or maintainability.
- Occurs after the system has been in place and running fine for a while, and end users start asking for minor tweaks improvements that could improve the way the system works.
- Examples:
 - Better input forms shortcut commands, better help system or error reporting, making the system more responsive.



Preventive Maintenance

- Proactively change the software to avoid future problems.
- Performed for the purpose of preventing problems before they occur.
- Examples:
 - Code reviews: Regular code reviews can be considered a form of preventive maintenance in software engineering as they help identify potential issues or defects in the code before they manifest into larger problems. Code reviews can help ensure that coding standards and best practices are being followed and that the code is maintainable.
 - Automated testing: Automated tests can help identify issues or defects in the software before they are released to users, reducing the likelihood of costly downtime or errors. Automated testing can also help ensure that software updates or changes do not negatively impact existing functionality.



3. Need for Maintenance

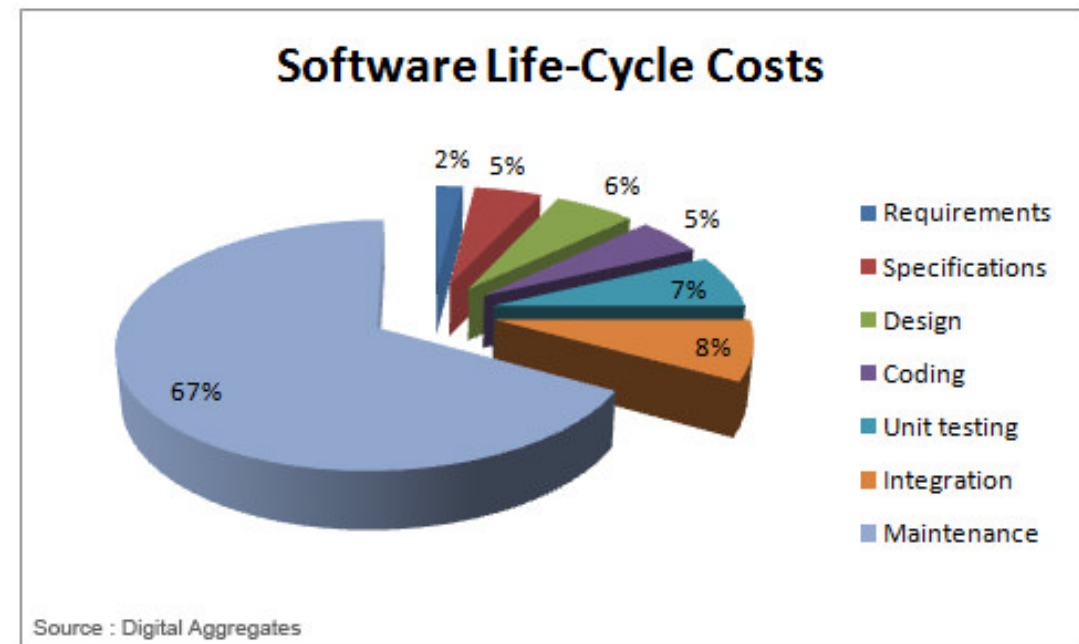
- Maintenance as a crucial and critical activity.
- We need to understand why software maintenance is necessary.
- **Importance of Software Maintenance**
 - Ensures optimal system performance by fixing bugs and errors that may occur during operation.
 - Helps keep software up-to-date with the latest technology and industry standards.
- **Benefits of Software Maintenance**
 - Increases system reliability and reduces the risk of system failures and downtime.
 - Helps identify and fix security vulnerabilities, ensuring that software is secure and protected against cyber threats.

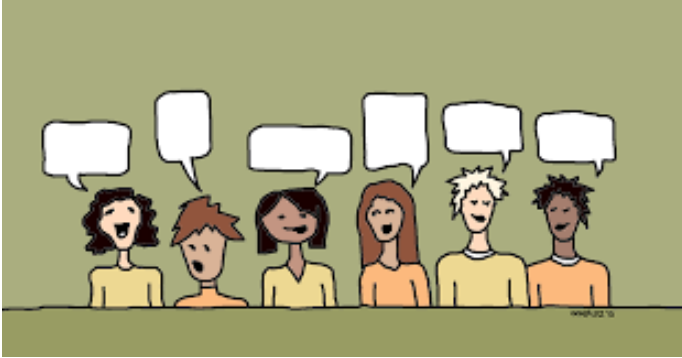
Cost Savings with Software Maintenance

- Reduces the need for costly system replacements by extending the lifespan of software.
- Increases efficiency and productivity by reducing the time and resources needed to fix software issues.

4. Majority of Maintenance Costs

- Usually greater than development costs (2 to 100 times depending on the application).
- This is due to various factors, including technical and non-technical factors
- The fact that maintenance often corrupts the software structure, making further maintenance more difficult and costly.





What are the factors that contribute to maintenance costs?

Open Class discussion (#2)

Factors that contribute to maintenance costs

- Changing User Requirements:
 - As users' needs and preferences change, software must be updated to remain relevant, which can add to maintenance costs
 - It is usually more expensive to add new features to a system during maintenance than it is to add the same features during development
 - A new team has to understand the programs being maintained
 - Separating maintenance and development means there is no incentive for the development team to write maintainable software
 - Program maintenance work is unpopular
 - Maintenance staff are often inexperienced and have limited domain knowledge.
- Aging software and hardware:
 - Older software is typically more expensive to maintain (e.g. old languages, compilers etc.).
 - their structure degrades and they become harder to change

Factors that contribute to maintenance costs

- Code Complexity:
 - Software that is complex and difficult to understand can be more costly to maintain, as it requires more time and effort to make changes and fixes
- Integration Challenges:
 - Software that integrates with other systems or applications can be more difficult and expensive to maintain, particularly if those systems undergo changes or updates

Topic 1: Software Maintenance

1. Definitions and Terminology
2. Nature of Maintenance
3. Need for Maintenance
4. Majority of Maintenance Costs
5. Evolution of Software
6. Categories of Maintenance

Importance of evolution

- Software systems is one of the critical business assets.
- To maintain the value of these assets to the business, they must be changed and updated.

Banking applications



Cyber threats

- Cyber attacks are becoming increasingly sophisticated, and businesses must evolve their software to protect against new types of attacks.
- For instance, ransomware attacks have become more prevalent, with cyber criminals using new techniques to evade detection and steal data.

Software change

- Software change is inevitable
 - New requirements emerge when the software is used;
 - The business environment changes;
 - Errors must be repaired;
 - New computers and equipment is added to the system;
 - The performance or reliability of the system may have to be improved.
- A key problem for all organizations is implementing and managing change to their existing software systems.

Software ages

- Software aging refers to the tendency for software to fail or cause system failures after running continuously for a certain period of time.
- **Reasons why software ages**
 - Maintenance activities
 - Ignorant surgery and architectural erosion
 - Inflexibility from the start
 - Insufficient or inconsistent documentation
 - Deadline pressure
 - Duplicated functionality (code duplication)
 - Lack of modularity



Maintenance vs. Evolution

- Software maintenance typically does not involve major changes to the system's architecture
- Changes are implemented by modifying existing components and adding new components to the system
- Software evolution is a broader term that encompasses both software maintenance and bigger changes.
- Evolution is intrinsic to the very nature of software development.
- At different phases of the software life-cycle

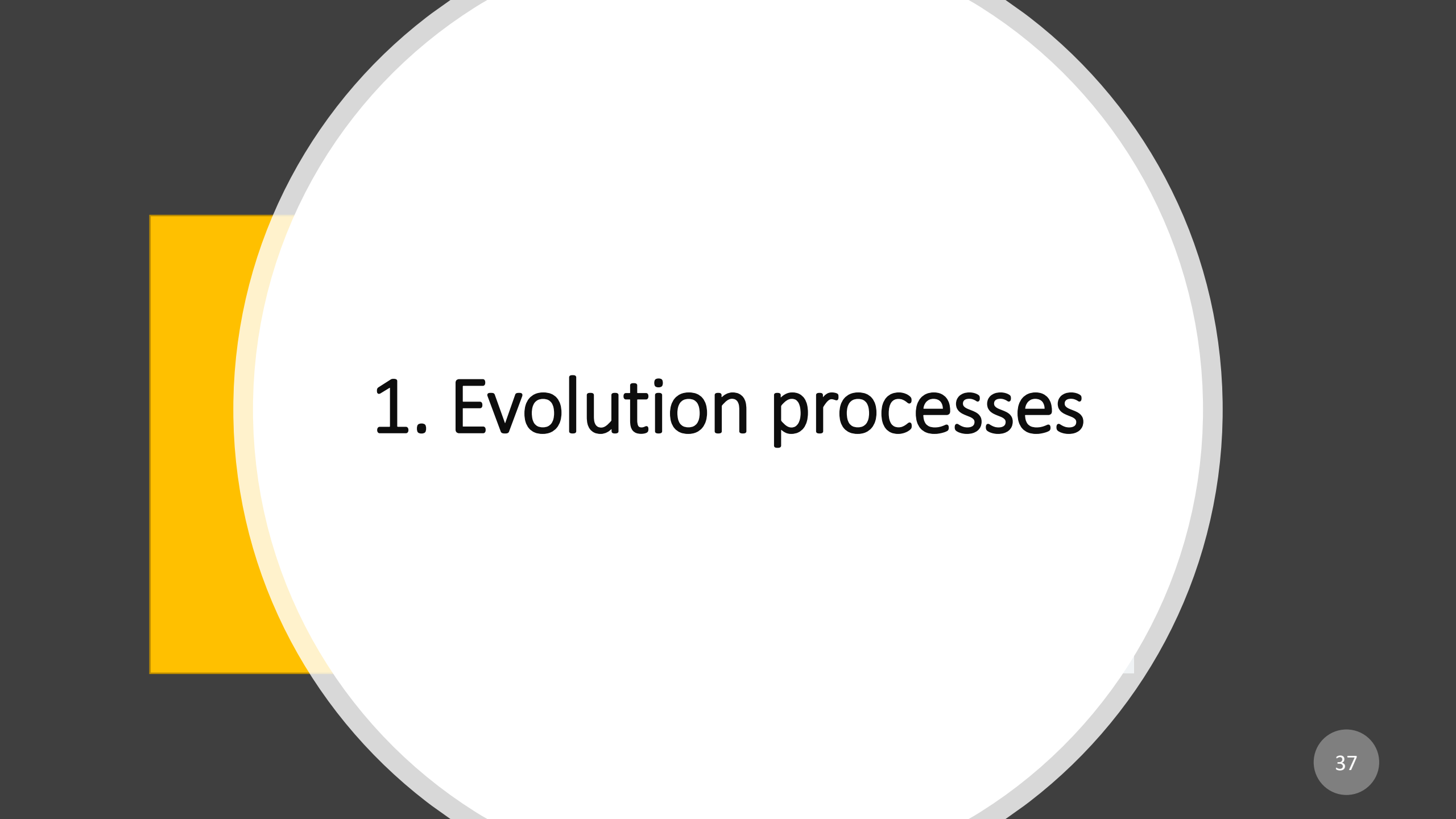
Software Evolution

- Evolution may arise
 - **During software development** – where the design evolves and matures as the understanding of the problem to be solved and how to solve it gradually increases
 - **During software maintenance** – after deployment, in the continuing process of adapting the system to be changing needs of its users and usage environment
 - When the system's continuous evolution made it too complex to maintain – the system may require a more extensive restructuring, redesign or even a full reimplementation or replacement.

Definition of Software Evolution

- Definition:

All programming activity that is intended to generate a new software version from an earlier operational version
[Lehman & Ramil, 2000]

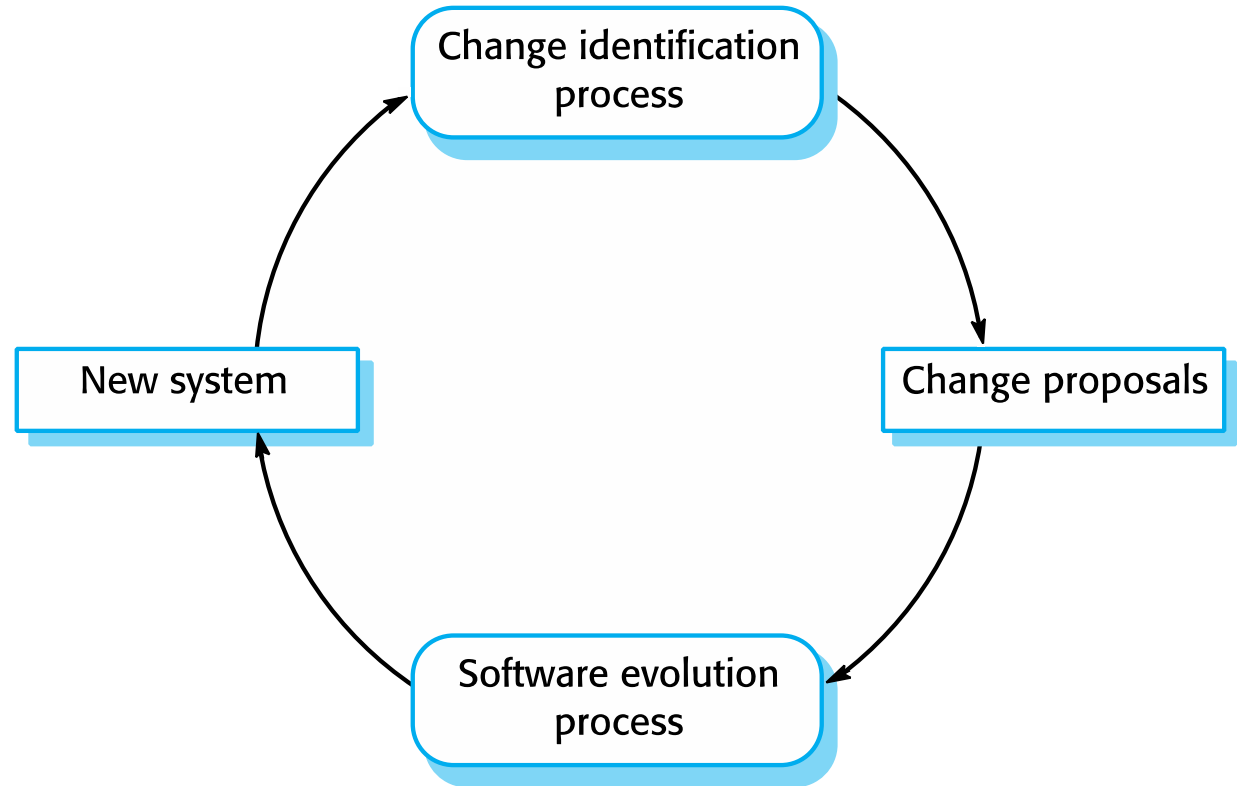


1. Evolution processes

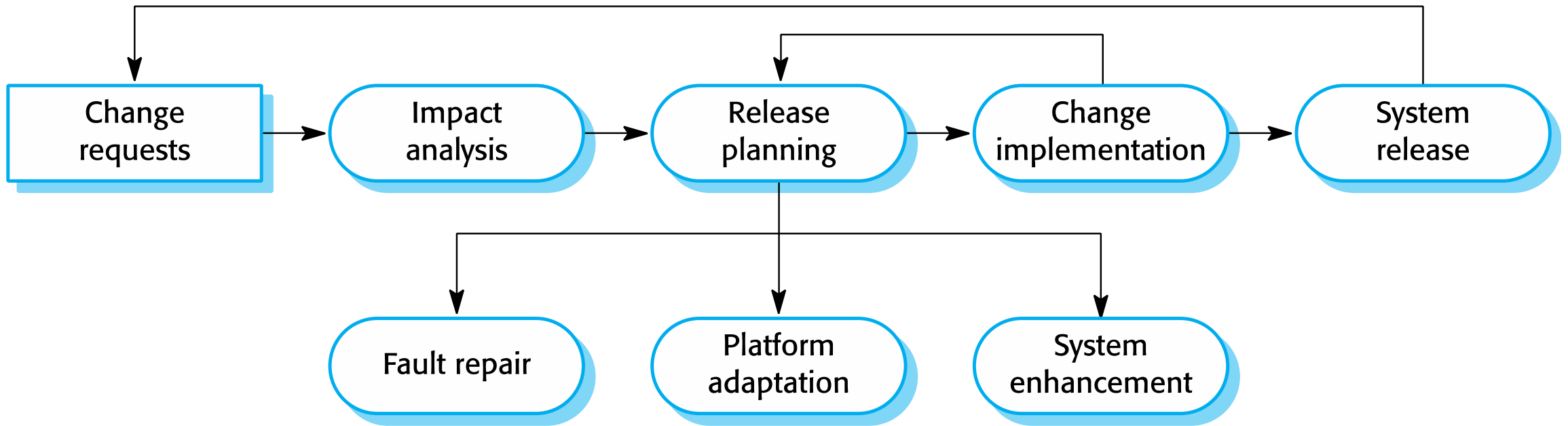
Evolution processes

- Software evolution processes depend on
 - The type of software being maintained;
 - The development processes used;
 - The skills and experience of the people involved.

Change identification and evolution processes



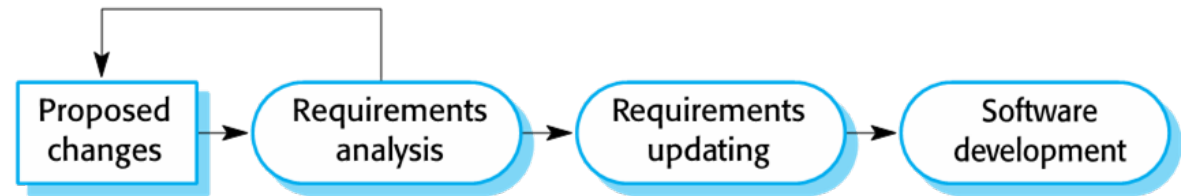
- The processes of change identification and system evolution are cyclical and continue throughout the lifetime of a system



The software evolution process

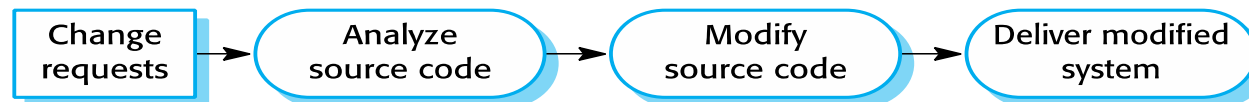
Change implementation

- Iteration of the development process where the revisions to the system are designed, implemented and tested.
- The first stage of change implementation involve program understanding
 - Important if the original system developers are not responsible for the change implementation.
 - understand how the program is structured, how it delivers functionality and how the proposed change might affect the program.



Urgent change requests

- Urgent changes may have to be implemented without going through all stages of the software engineering process
 - If a serious system fault has to be repaired to allow normal operation to continue;
 - If changes to the system's environment (e.g. an OS upgrade) have unexpected effects;
 - If there are business changes that require a very rapid response (e.g. the release of a competing product).



Agile methods and evolution

- Agile methods are based on incremental development so the transition from development to evolution is a seamless one.
 - Evolution is simply a continuation of the development process based on frequent system releases.
- Automated regression testing is particularly valuable when changes are made to a system.
- Changes may be expressed as additional user stories.

Program Evolution Dynamics – Lehman's Laws

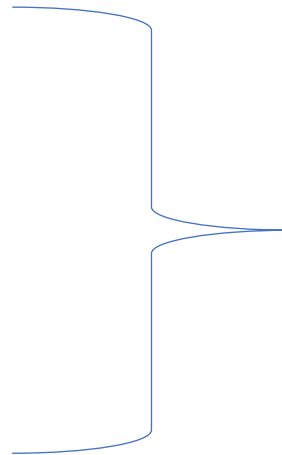
Law	Description
Continuing change	A program that is used in a real-world environment must necessarily change, or else become progressively less useful in that environment.
Increasing complexity	As an evolving program changes, its structure tends to become more complex. Extra resources must be devoted to preserving and simplifying the structure.
Large program evolution	Program evolution is a self-regulating process. System attributes such as size, time between releases, and the number of reported errors is approximately invariant for each system release.
Organizational stability	Over a program's lifetime, its rate of development is approximately constant and independent of the resources devoted to system development.
Conservation of familiarity	Over the lifetime of a system, the incremental change in each release is approximately constant.
Continuing growth	The functionality offered by systems has to continually increase to maintain user satisfaction.
Declining quality	The quality of systems will decline unless they are modified to reflect changes in their operational environment.
Feedback system	Evolution processes incorporate multiagent, multiloop feedback systems and you have to treat them as feedback systems to achieve significant product improvement.

Topic 1: Software Maintenance

1. Definitions and Terminology
2. Nature of Maintenance
3. Need for Maintenance
4. Majority of Maintenance Costs
5. Evolution of Software
6. Categories of Maintenance

6. Categories of Maintenance

- Important to understand the different techniques of software maintenance
- Categories of Maintenance:
 - Legacy Systems,
 - Reengineering,
 - Impact Analysis,
 - Refactoring,
 - Program Comprehension



TECHNIQUES

Legacy systems



Legacy Systems

- How do organizations address the challenges posed by legacy systems, such as outdated software applications and hardware, in today's rapidly evolving technological landscape, while ensuring continuity of business operations and compliance with modern security standards?

What is a Legacy System?



What's from ChatGPT

Q: Provide three latest examples of legacy systems related to software maintenance.



Based on the provided search results, here are three examples of legacy systems related to software maintenance:



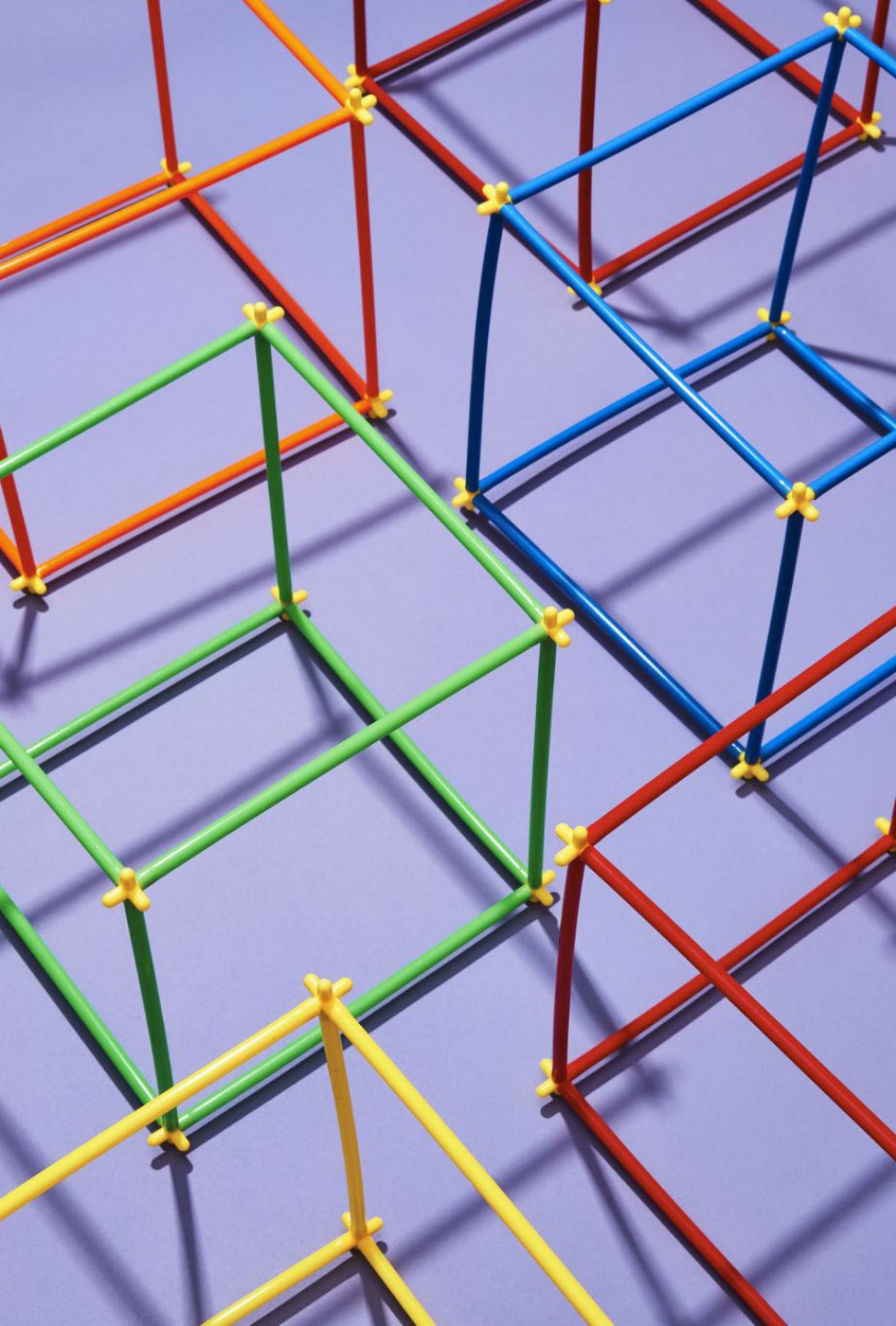
1. Oracle, Microsoft, and IBM software products with end-of-life and end-of-support status (2018) [2]. Many organizations still use these products despite their outdated status, which can create challenges in terms of maintenance and support.
2. Other examples of legacy systems can include any outdated software or hardware still in use that may lack vendor support or fail to meet an organization's evolving requirements [1]. This can include anything from old custom-built applications to aging ERP systems.
3. Software maintenance itself can also be considered a legacy system, as the traditional approach of software maintenance is no longer sustainable due to the high costs and risks involved [3]. This has led to the development of new approaches to software maintenance, such as continuous software delivery and DevOps practices, which aim to reduce costs and increase efficiency.

Overall, these examples demonstrate the challenges that organizations face when dealing with legacy systems related to software maintenance, including outdated software products, custom-built applications, and traditional software maintenance approaches. To address these challenges, it is essential for organizations to adopt modern approaches and technologies to maintain their software systems effectively.

🔄 Regenerate response

Press / to open the commands menu





Legacy systems

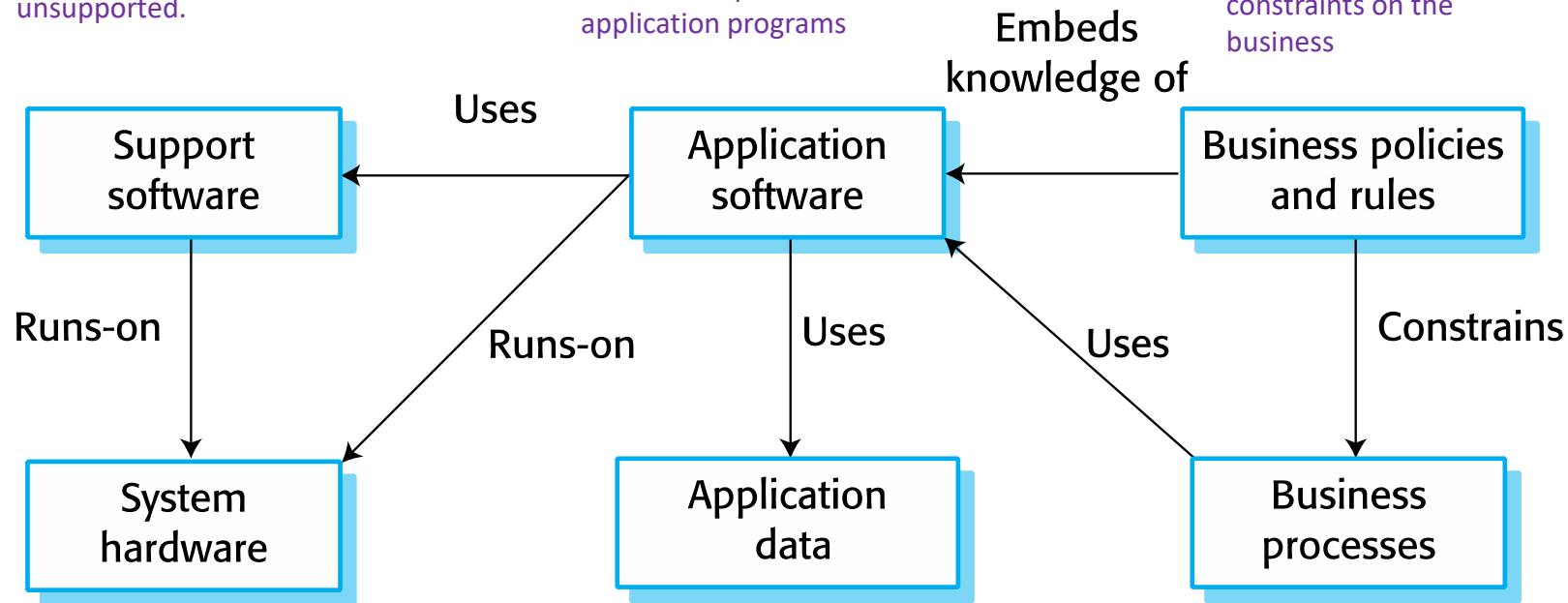
- Legacy systems are older systems that rely on languages and technology that are no longer used for new systems development.
- Legacy software may be dependent on older hardware, such as mainframe computers and may have associated legacy processes and procedures.
- Legacy systems are not just software systems but are broader socio-technical systems that include hardware, software, libraries and other supporting software and business processes.

The elements of a legacy system

Legacy system may rely on a range of support software, which may be obsolete or unsupported.

The application system that provides the business services that made up of a number of application programs

Definitions of how the business should be carried out and constraints on the business



Legacy systems may have been written for hardware that is no longer available

Data that are processed by the application system, may be inconsistent, duplicated or held in different databases.

Processes that are used in the business to achieve some business objective.

Legacy system replacement

- System replacement is risky and expensive for a number of reasons
 - Lack of complete system specification
 - Tight integration of system and business processes
 - Undocumented business rules embedded in the legacy system
 - New software development may be late and/or over budget

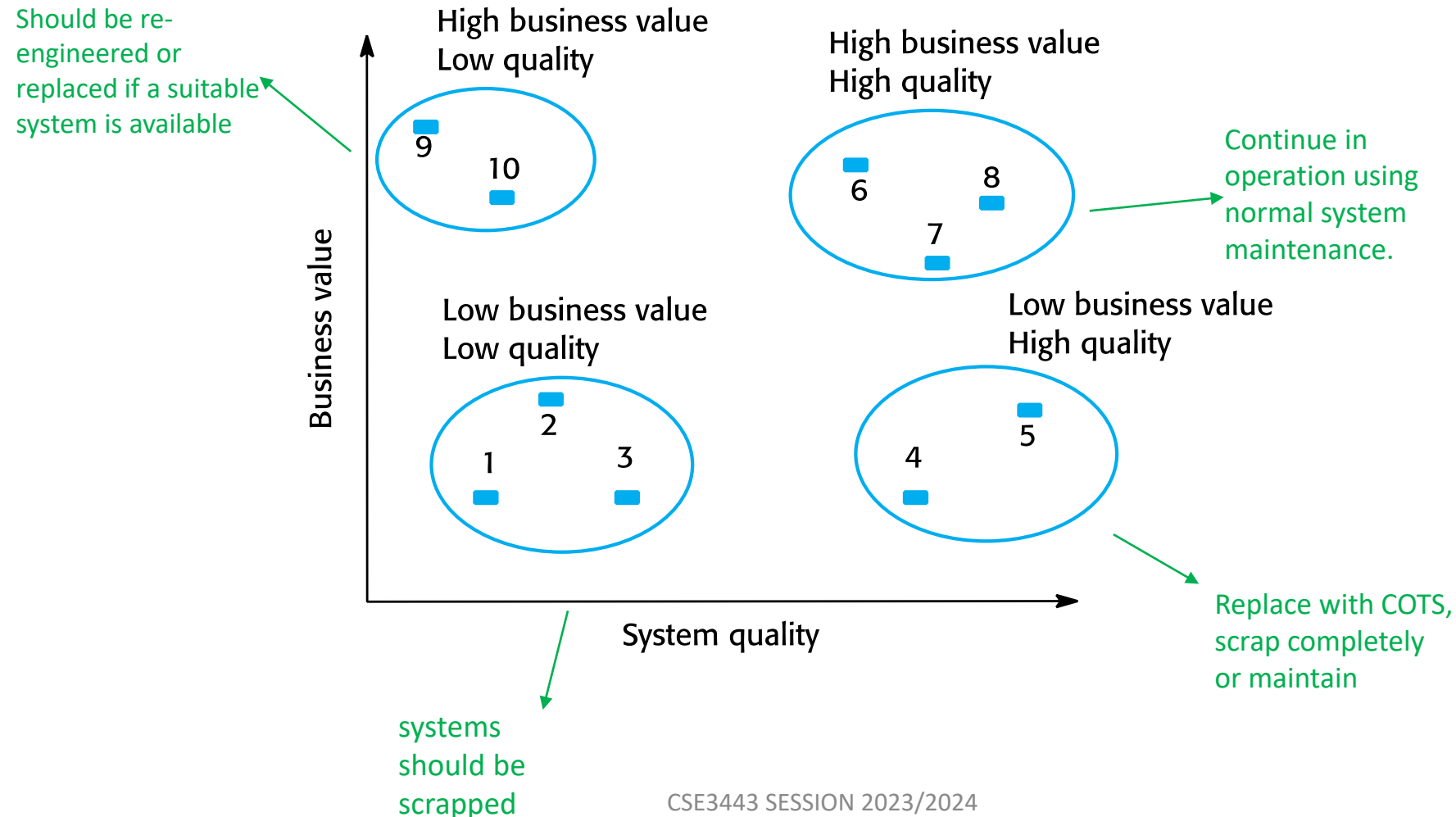
Legacy system change

- Legacy systems are expensive to change for a number of reasons:
 - No consistent programming style
 - Use of obsolete programming languages with few people available with these language skills
 - Inadequate system documentation
 - System structure degradation
 - Program optimizations may make them hard to understand
 - Data errors, duplication and inconsistency

Legacy system management

- Organizations that rely on legacy systems must choose a strategy for evolving these systems
 - Scrap the system completely and modify business processes so that it is no longer required;
 - Continue maintaining the system;
 - Transform the system by re-engineering to improve its maintainability;
 - Replace the system with a new system.
- The strategy chosen should depend on the system quality and its business value.

An example of a legacy system assessment



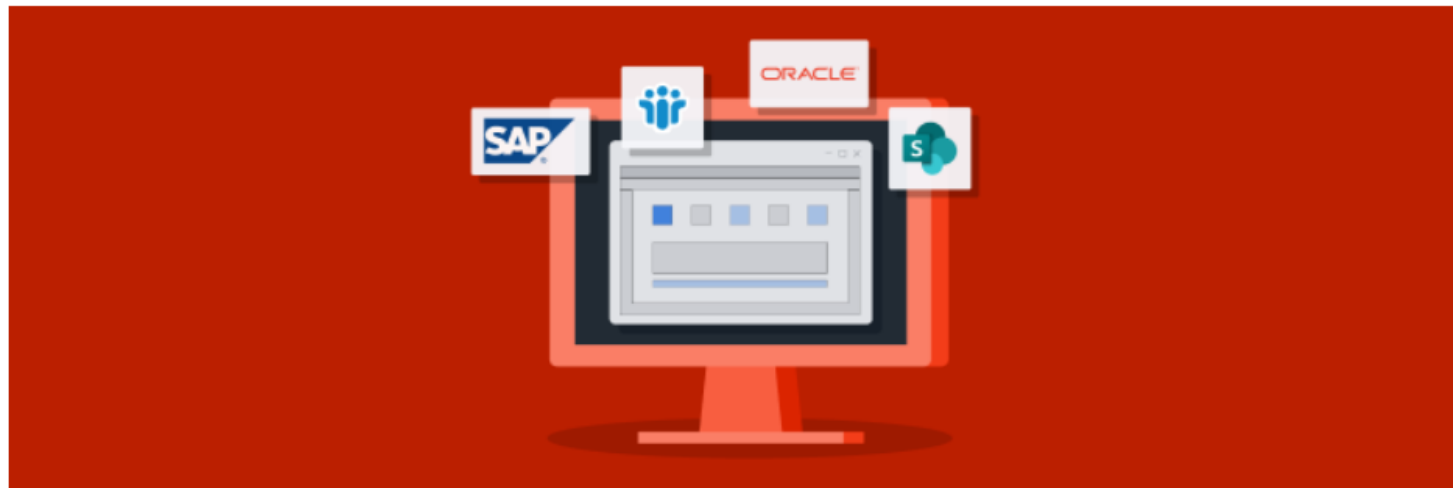
4 Examples of Legacy Systems (And How to Modernize Them)



Tony Ollivier

June 20, 2022 • 4 min read

A legacy system is any outdated software or hardware that it's still in use. Although the term may imply it, a legacy system isn't necessarily an aged technology. It can be a system that lacks vendor support or doesn't meet an organization's evolving requirements. In this blog post, we'll cover four examples of legacy systems that companies often struggle to modernize.



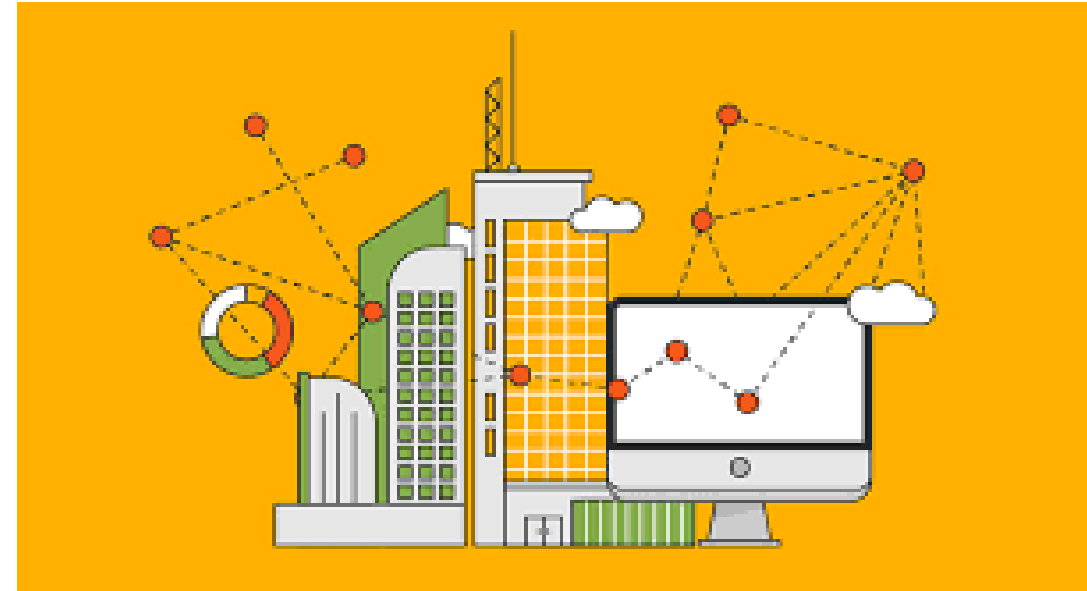


Software reengineering

- Restructuring or rewriting part or all of a legacy system without changing its functionality.
- Applicable where some but not all sub-systems of a larger system require frequent maintenance.
- Reengineering involves adding effort to make them easier to maintain. The system may be re-structured and re-documented.

Reengineering

- How can businesses effectively leverage software reengineering to modernize their legacy systems and improve the maintainability and performance of their software applications in today's dynamic technological landscape?





Advantages of reengineering

Reduced risk

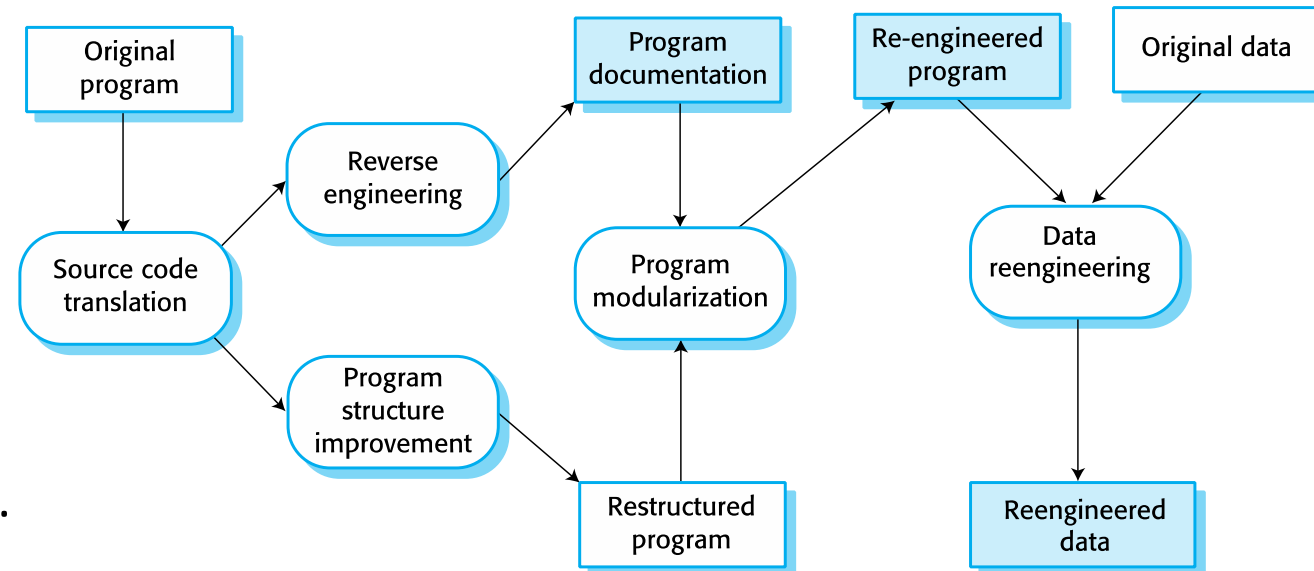
- There is a high risk in new software development. There may be development problems, staffing problems and specification problems.

Reduced cost

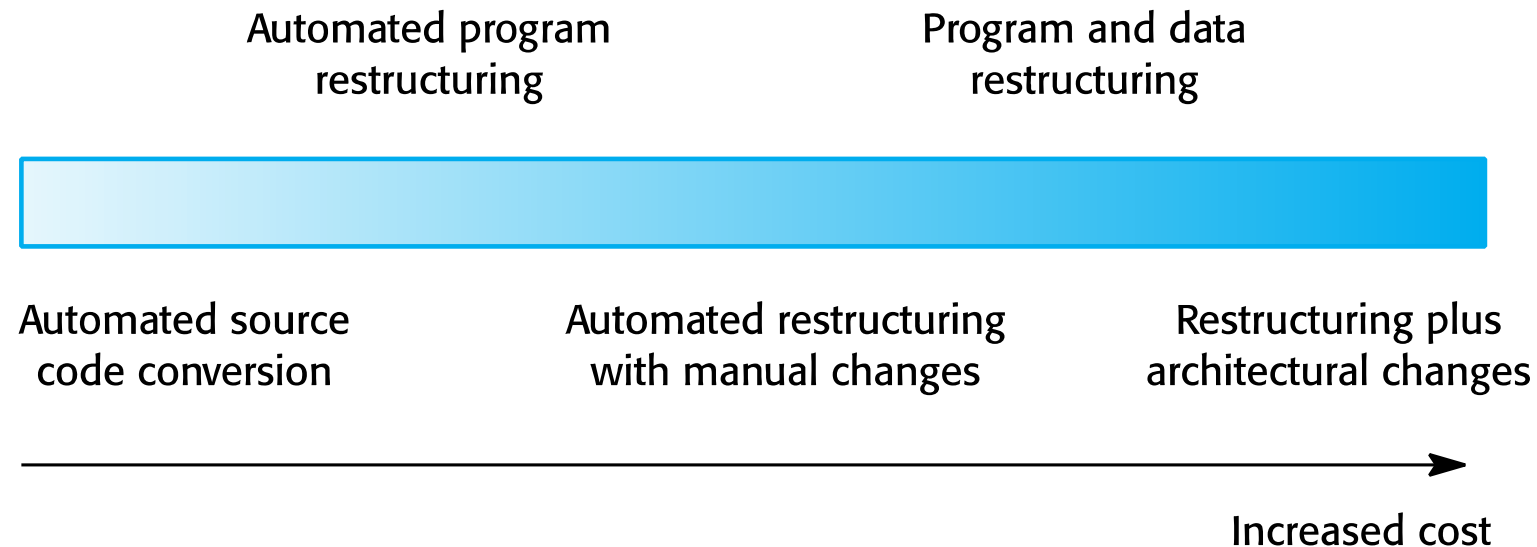
- The cost of re-engineering is often significantly less than the costs of developing new software.

The reengineering process

- Source code translation
 - Convert code to a new language.
- Reverse engineering
 - Analyse the program to understand it;
- Program structure improvement
 - Restructure automatically for understandability;
- Program modularisation
 - Reorganise the program structure;
- Data reengineering
 - Clean-up and restructure system data.



Reengineering approaches





Reengineering cost factors

- The quality of the software to be reengineered.
- The tool support available for reengineering.
- The extent of the data conversion which is required.
- The availability of expert staff for reengineering.
 - This can be a problem with old systems based on technology that is no longer widely used.

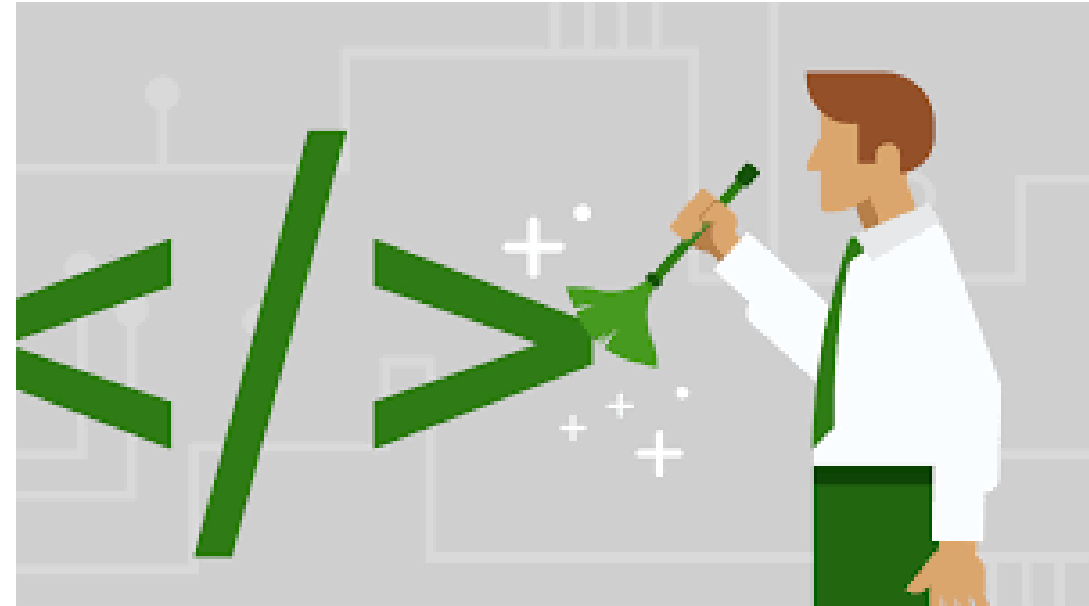
Refactoring

- Refactoring is the process of making improvements to a program to slow down degradation through change.
- A 'preventative maintenance' that reduces the problems of future change.
- Refactoring involves modifying a program to improve its structure, reduce its complexity or make it easier to understand.
- Did not add functionality but rather concentrate on program improvement



Code Refactoring

- In what ways can software developers utilize code refactoring to enhance the maintainability and readability of a complex e-commerce platform's codebase while ensuring uninterrupted functionality and improving overall system performance?



Refactoring and reengineering

Re-engineering

- takes place after a system has been maintained for some time and maintenance costs are increasing.
- use automated tools to process and re-engineer a legacy system to create a new system that is more maintainable.

Refactoring

- a continuous process of improvement throughout the development and evolution process.
- Purpose to avoid the structure and code degradation that increases the costs and difficulties of maintaining a system.

Key Challenges Faced by Practitioners in Software Evolution and Maintenance

- Legacy Code:
 - One of the biggest challenges in software maintenance is dealing with legacy code that may be difficult to understand, modify, or integrate with new systems.
- Technical Debt:
 - Technical debt refers to the accumulated cost of short-term solutions that were implemented in the past, which can slow down maintenance efforts and increase the risk of errors.
- Knowledge Management:
 - Maintaining software requires a deep understanding of the code, the system architecture, and the business requirements, which can be difficult to manage over time.
- Security Risks:
 - As software evolves, it may become vulnerable to new security threats, and maintaining security requires ongoing efforts to identify and mitigate risks.
- End-User Expectations:
 - End-users may have high expectations for software performance, functionality, and usability, and maintaining software requires balancing these expectations with the realities of limited resources and competing priorities.

Change induces technical debt

- As a change happens, often there is a need to make other coordinated changes at the same time in other parts of the software.
- These other required changes, if not completed immediately, incur a kind of debt that must be paid at some point in the future.
- This technical debt is the extra development work that arises when code that is easy to implement in the short run is used instead of applying the best overall solution.
- Just like financial debt, if technical debt is not repaid, the uncompleted changes accumulate interest on top of interest, because of increased entropy.
- The longer you wait to make the changes, the harder it becomes.

answers for each subject.

Query: contrast between software reengineering and refactoring, in point forms



Here are some key points that contrast software reengineering and refactoring:



Software Reengineering:

- Involves fundamental changes to a program's structure and design [2]
- Can affect the entire system or part of the system and may result in a new system being created [3]
- Typically done to improve the maintainability, performance, and quality of a software system [1]
- May involve sub-processes like reverse engineering, restructuring, and forward engineering [3]

Refactoring:

- Involves changing bits and pieces of software while keeping the core of a program mostly untouched [2]
- Improves the internal structure of the code without altering the external behavior [1]
- Typically done to improve the quality, maintainability, and readability of the code [1]
- May involve techniques like simplification, code reuse, and optimization [2]

Open Discussion

Topic coverage:

- Maintenance issues
- The need for maintenance
- Types of maintenance
- Evolution of software
- Techniques of software maintenance



Question 1: What are some common causes of maintenance issues in industrial settings?

- i. Collaborate with each other to find the answers – use ChatGPT

Some examples of how to present your answers to the question.

	Legacy systems	Reengineering	Refactoring
What			
Why			
How			
Examples			



Question 2: Why is maintenance important in ensuring software reliability?

- i. Collaborate with each other to find the answers – use ChatGPT

Some examples of how to present your answers to the question.

	Legacy systems	Reengineering	Refactoring
What			
Why			
How			
Examples			



Question 3: Explain the concept of corrective maintenance and provide examples?

- i. Collaborate with each other to find the answers – use ChatGPT

Some examples of how to present your answers to the question.

	Legacy systems	Reengineering	Refactoring
What			
Why			
How			
Examples			



Question 4: How does adaptive maintenance differ from other types of maintenance?

- i. Collaborate with each other to find the answers – use ChatGPT

Some examples of how to present your answers to the question.

	Legacy systems	Reengineering	Refactoring
What			
Why			
How			
Examples			



Question 5: What is preventive maintenance, and why is it crucial in asset management?

- i. Collaborate with each other to find the answers – use ChatGPT

Some examples of how to present your answers to the question.

	Legacy systems	Reengineering	Refactoring
What			
Why			
How			
Examples			



Question 6: How has software evolved over time, leading to the need for various maintenance techniques?

- i. Collaborate with each other to find the answers – use ChatGPT

Some examples of how to present your answers to the question.

	Legacy systems	Reengineering	Refactoring
What			
Why			
How			
Examples			



Question 7: Can you describe the concept of legacy systems in software engineering?

- i. Collaborate with each other to find the answers – use ChatGPT

Some examples of how to present your answers to the question.

	Legacy systems	Reengineering	Refactoring
What			
Why			
How			
Examples			



Question 8: What is reengineering in the context of software maintenance, and why is it necessary?

- i. Collaborate with each other to find the answers – use ChatGPT

Some examples of how to present your answers to the question.

	Legacy systems	Reengineering	Refactoring
What			
Why			
How			
Examples			



Question 9: How does refactoring contribute to software maintenance and improvement?

- i. Collaborate with each other to find the answers – use ChatGPT

Some examples of how to present your answers to the question.

	Legacy systems	Reengineering	Refactoring
What			
Why			
How			
Examples			



Question 10: What are some real-world examples where software refactoring has led to significant improvements?

- i. Collaborate with each other to find the answers – use ChatGPT

Some examples of how to present your answers to the question.

	Legacy systems	Reengineering	Refactoring
What			
Why			
How			
Examples			

